

RBE 4701 – AI for Robotics

Project 2: Approximate Q-Learning Bomberman Agent

Team 03: Bryce Lukacs & Benni Valentine

Instructor: Prof. Jing Xiao

Date: October 8, 2025

Repo: [Github Link Here](#)

Abstract

For this project we built a Bomberman AI that uses **Approximate Q-Learning** along with an **A*** path planner. The goal was to create a character that could survive and reach the exit in all five map variants while handling monsters, walls, and bombs. Our agent learns using a small set of features that describe its situation and updates its weights every time step, while also using A* to plan safe routes, especially when a bomb goes off or a monster gets close. After tuning rewards and fixing wall detection, our agent now passes every variant with a 10/10 success rate. It usually picks safer routes instead of the fastest ones, but with enough training, it almost never dies early and finds the exit.

1. Structure of our approach

1.1 Overview

Our agent combines two ideas: a basic Q-learning system and a safety/longevity planner. Each turn, it checks if A* can find a safe route to the exit, and if so, it follows that route. If not, it uses its learned Q-values to pick the best next move. When it plants a bomb, it immediately switches to an “escape mode” that finds a clear path away from the blast. We had issues with it blowing itself up a ton so we HEAVILY (I believe we have a penalty of 10,000) weighted not killing itself and that made our success rates go up exponentially. If no good moves exist, it waits instead of running into danger. This setup keeps the agent from freezing or making random moves when things get chaotic and makes our runs a lot more consistent.

1.2 Features and Actions

The character looks at seven things about the world each tick:

- How close it is to the exit
- If it's still alive

- How many walls are left
- Whether a bomb is active
- How soon a nearby bomb will explode
- How far it is from the nearest bomb or blast
- How far it is from any monsters

It can move in eight directions or place a bomb, giving it nine possible actions. All features are normalized so that none of them overpower the others when learning.

2. Implementation Details

2.1 Q-Learning Setup

We used simple linear weights for each feature, and the agent keeps them in a `weights.csv` file so it can remember what it's learned between runs. Each move updates the weights slightly based on how good or bad the outcome was, and if the next step would kill the player, we immediately assign a big negative value to stop it from doing that again (we had instances where the agent would just do the same terrible path over and over).

2.2 Reward Tuning

We adjusted the rewards until the character played smart and consistent. It earns smaller rewards for moving closer to the exit and surviving, but we have big penalties for dying. It also gets a bonus when a bomb is placed (since breaking walls and killing monsters help us eventually), but it's also rewarded for backing away from the explosion. Extra distance from monsters adds small positive rewards so it keeps space and doesn't get blitzed and die. This balance made the character cautious and took a little longer to reach the goal but it is incredibly reliable.

2.3 A* Planner

The built-in A* planner gives the agent a safety net. It looks for a clear route to the exit or to a safe zone after dropping a bomb. We also allow diagonal moves so paths are smoother and quicker.

2.4 Practical Notes

- All dependencies are standard: NumPy, Pygame, and PyTorch
- The agent automatically saves weights every episode.

3. Experimental Results

3.1 Setup

All tests were done on Windows 11 using Python 3.12 and Pygame 2.6.1. Each variant was run 10 times. The same Q Learning character was used for all maps, so we had no variant specific tuning.

3.2 Results

Variant	Wins (out of 10)	Notes
1	10 / 10	Reaches exit quickly
2	10 / 10	Avoids monsters cleanly
3	10 / 10	Handles bombs really well
4	10 / 10	Navigates tight corridors using bombs
5	10 / 10	Mixed hazards; smooth and consistent, takes a little more time to reach exit but prioritizes not dying

Overall: 100% success across all variants. (we died twice across 114 runs in my most recent test) The agent consistently reaches the goal without deaths. It prioritizes safety so it will sometimes go a roundabout way to safely reach the exit but it never gets hard stuck, it just keeps positioning until it's able to reach the goal.

3.3 Behavior Summary

- Always escapes blast zones in time
- Drops bombs only when it has a safe way out
- Breaks walls when needed but doesn't overuse bombs
- It waits patiently if moving would be dangerous
- No longer gets stuck or paces in corners (this was a problem we were running into at the start of the project)

4. Discussion

The final version focuses heavily on survival rather than speed. This trade-off makes it slower but extremely reliable compared to our project 1. It doesn't always choose the most direct path, but it never makes suicidal moves to prioritize speed. We found the wavefront features and explosion checks especially helpful for keeping the AI aware of future danger.